

MTE 484

Digital Control Systems

Ball and Beam Lab Project

Student Handout

CONTENTS

- General Safety Precautions
- Notes on Lab Group Work
- Lab Overview
- Lab 1: Modeling of the Motor System Plant
- Lab 2: Design of the Inner-Loop Controller and Modeling of the Ball and Beam
- Lab 3: Design of the Outer-Loop Controller
- Demo: Demonstration of the Controlled Ball & beam System + Q&A
- Appendix 1: Ball and Beam Datasheet
- Appendix 2: Discrete Controller Implementation

GENERAL SAFETY PRECAUTIONS

All students enrolled in this course received the access code to the door lock. This code must not be shared with anyone not currently enrolled in the course.

Access to the lab can be made 24/7, however exercise caution, especially when working outside of regular “business hours” (8:00 AM 5:00 PM):

- a) Avoid working alone and never prop any of the doors open.
- b) If working alone, have your mobile phone available and charged, and let at least one person know of your location.
- c) Pay 100% attention to the work you are doing.
- d) In this lab, we work with rotating equipment, so be careful with long hair, jewelry, long sleeves and/or scarves to avoid entanglement with system gears.

NOTES ON LAB GROUP WORK

1. You and your lab partner should work together on all aspects of the lab, but you are not allowed to do significant work with any other students in the class. For example, you may talk with students in other groups about minor aspects of the lab (“I forgot—how do you start the Arduino Software?”; “Where did you find the published value of the ball’s mass?”), but you are not allowed to write up lab reports jointly with other lab groups or to share computer code in any way.
2. Lab group partners can define the dynamics of their work in group, but both partners need to be able to discuss/explain any aspect of the project if and when required.
3. Do not obtain or look at lab reports (either in hardcopy or softcopy) written by students in another group, whether they are taking MTE 484 / ECE 484 / ECE 481 presently or have taken it in a previous term.
4. In your report, you must completely and unambiguously acknowledge and reference any person, website, report, book, or notes that you used to help you with your work. You should reference this lab manual and the course notes, for example.

References and citations should be provided according to the IEEE Reference Guide (2018). See IEEE. "IEEE REFERENCE GUIDE", <https://ieeauthorcenter.ieee.org/wp-content/uploads/IEEE-Reference-Guide.pdf> (accessed Sept 26, 2022) for reference.

5. You must not let any student, except for your lab group partner, access any part of your lab report (either in hardcopy or softcopy).
6. In each lab report, include and sign, the following statement:

We acknowledge and promise that:

- (a) We are the sole authors of this lab report and associated simulation files/code.
- (b) This work represents our original work.
- (c) We have not shared detailed analysis or detailed design results, computer code, or Simulink diagrams with any other student.
- (d) We have not obtained or looked at lab reports from any other current or former student of ECE 481/ ECE 484 / MTE 484, and we have not let any other student access any part of our lab work.
- (e) We have completely and unambiguously acknowledged and referenced all persons and aids used to help us with our work.

✓ **Student 1 Name:** _____

✓ **Student 1 Signature*:** _____

✓ **Student 2 Name:** _____

✓ **Student 2 Signature*:** _____

✓ **Lab Group #:**

✓ **Station #:**

() We require an actual signature: a typed name is not an acceptable substitute.*

LAB OVERVIEW

This project deals with a classical control experiment where the objective is to move and position a ball on a beam by appropriately inclining the beam. Figure 1 shows the ball and beam apparatus that we will be using in this lab. As seen in the figure, a DC motor is used to control a lever arm, which in turn is used to raise or lower the right side of the beam to cause the ball to move. The beam itself consists of two parallel rods, which together with the ball form a potentiometer used to measure the ball's position. One of the rods has a resistive film glued onto it, while the other rod and the ball form the wiper of the potentiometer. The left side of the beam is pivoted on a support. The input to the ball and beam apparatus is the motor voltage; both the ball position and the motor gear angle are measurable outputs.

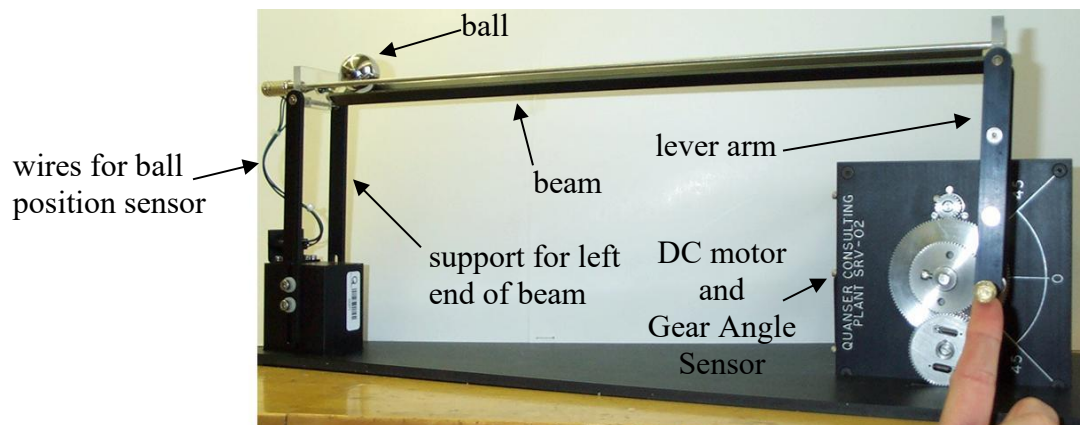


FIGURE 1: The plant: a “ball and beam” apparatus.

In this lab course, you will design and implement a discrete time control system **to move the ball back and forth along the beam**, between a position of 0.1 m from the left end of the beam to a position at 0.25 m from the left end of the beam.

To implement this control system, you will use an Arduino Controller box which includes analog to digital and digital to analog converters and a DC motor drive and amplifier.

The performance of the system will need to meet specifications given for example maximum allowed overshoot, settling time and steady state error.

Figure 2 shows the main connections between the ball and beam apparatus, Arduino Controller box and the lab Nexus computer.

It is strongly recommended that you use your own personal computer to perform the lab experiments, in which case the Arduino IDE will need to be downloaded and installed in your personal computer. This removes any conflict regarding administrative rights on the computer and safeguards against data loss due to university policies regarding data storage on the computer stations along the term.

The lab station computers already have the Arduino IDE installed and are an option as well.

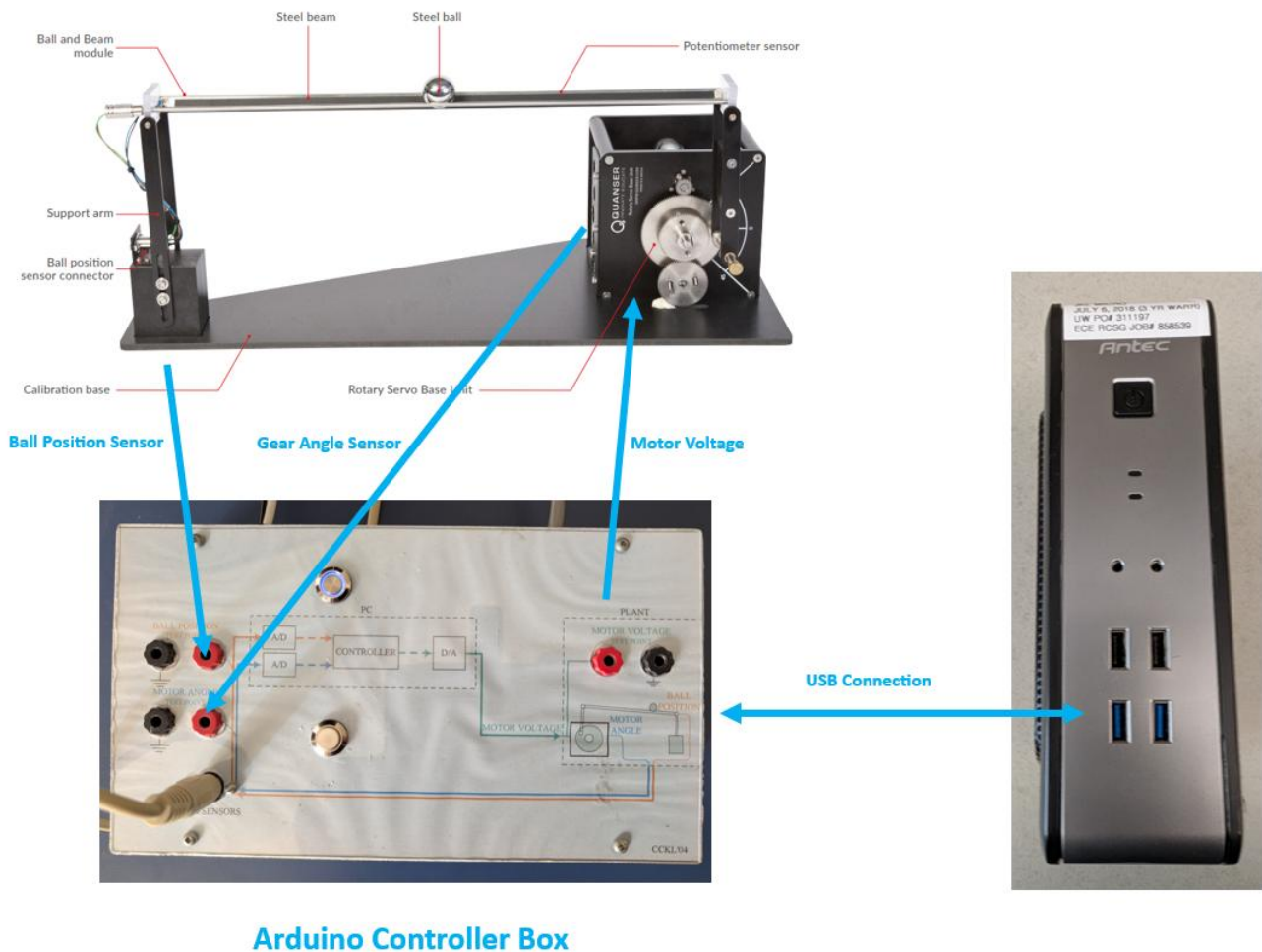


FIGURE 2: Key connections between the ball and beam apparatus, Arduino Controller and lab Nexus computer

Following link provides an 8-minute [video](#) showing the lab project in action.

An Arduino application (Arduino sketch) will need to be written by each lab group to control the ball and beam system at their assigned lab station.

An Arduino Library and initial sample code are available for download from the course website on LEARN.

The sample code provided allows you to send a voltage to the motor to obtain rotation, and also reads the positions of the gear angle and ball position potentiometers. The sample code provided also implements a periodic interrupt routine which you can use to generate the sampling interval(s) for your sampled time control system.

To setup your software working environment **on the lab station computer**, you will need to login to your assigned lab station (based on your lab group number) using your WAT ID credentials.

Then follow the steps below:

- 1) Navigate to: This PC -> Windows (C:) -> Users -> ***your WAT ID ***
- 2) Find the “Documents” Icon and right click on it -> Pin to quick access (This will allow you to find the code easily in the future)
- 3) Set up the Arduino folder structure as shown in Figure 3 below using the files/folders downloaded from the course website on LEARN.

If using your own personal computer, setup an Arduino folder under your Documents folder in your Quick Access links. Under this Arduino folder you will store the Libraries folder and the sample_code folder downloaded from LEARN.

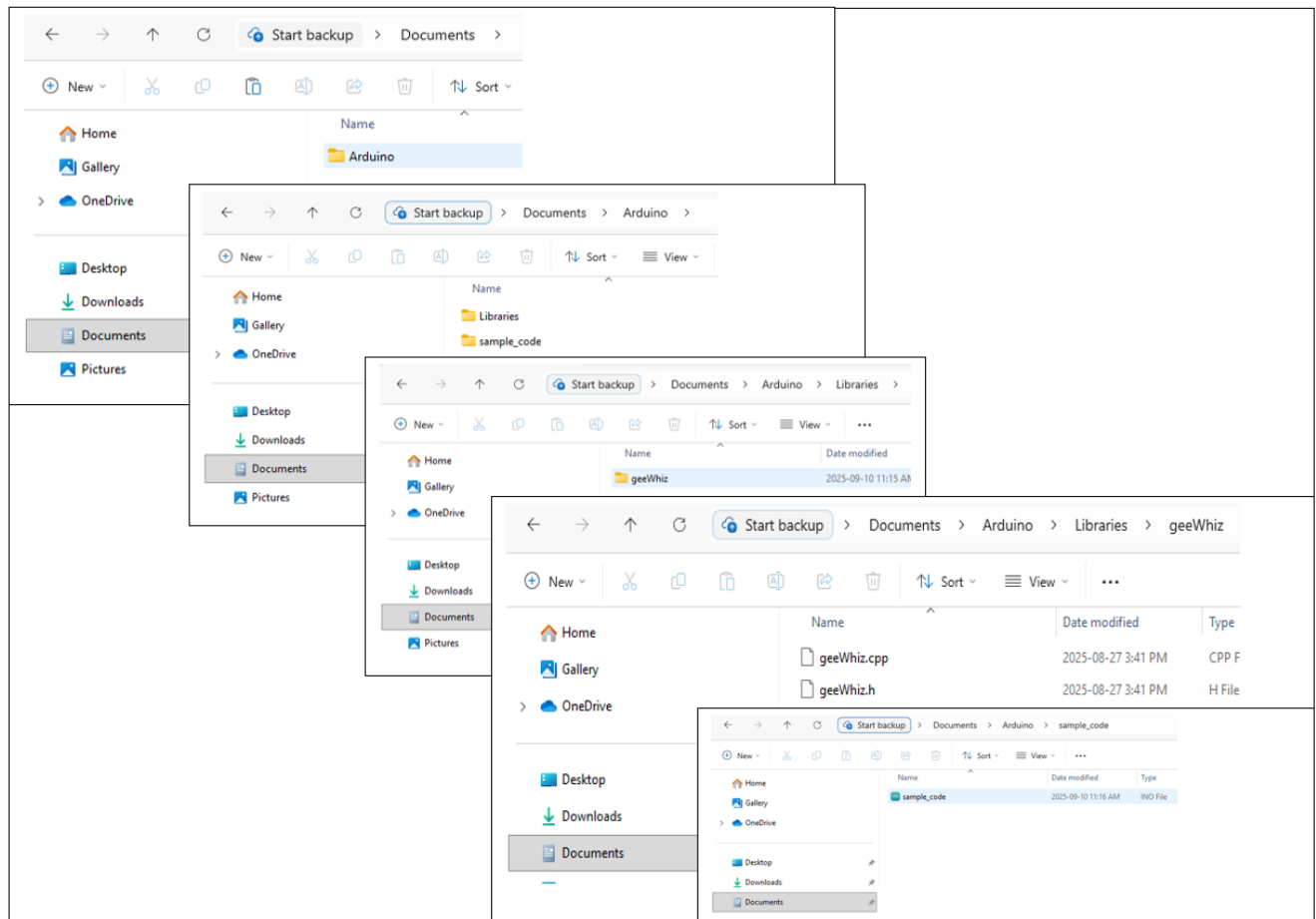
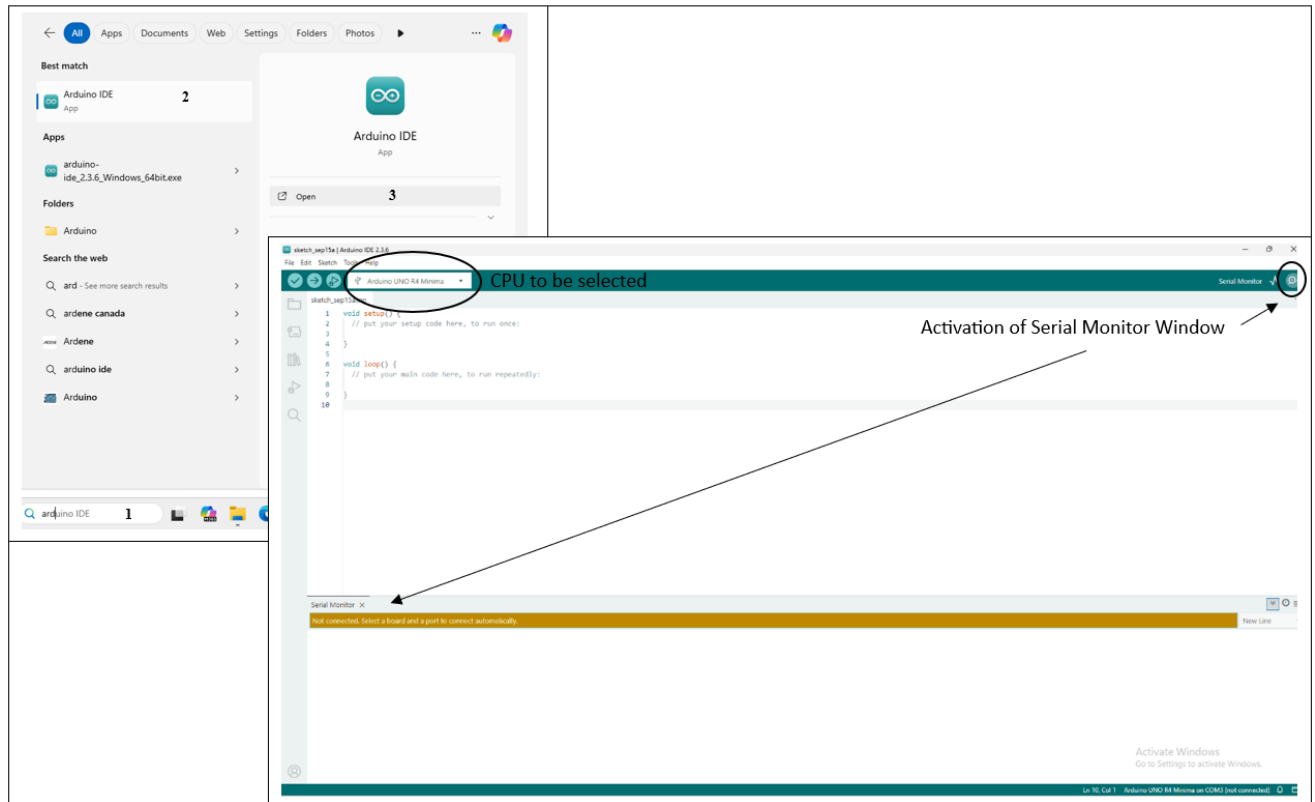


FIGURE 3: Arduino Folder Structure

The Arduino controller will be programmed using the Arduino IDE.



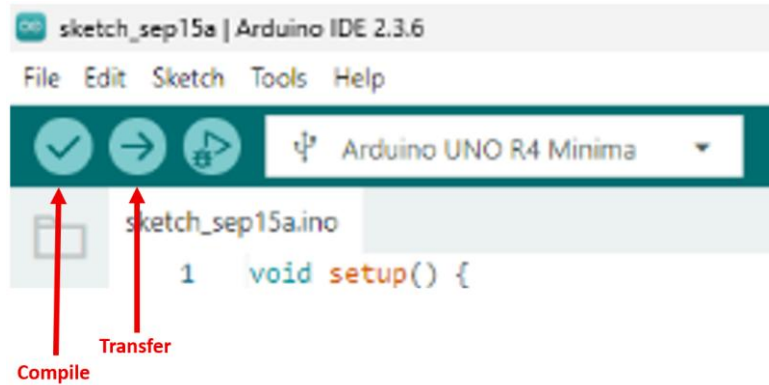
In the Arduino IDE, select CPU Arduino UNO R4 Minima, and program debugging can be done using the Serial Monitor, including `Serial.print()` and `Serial.println()` commands in your code to monitor variables during program execution.

The Arduino UNO R4 Minima boards used on the lab stations do not have a built-in debug probe.

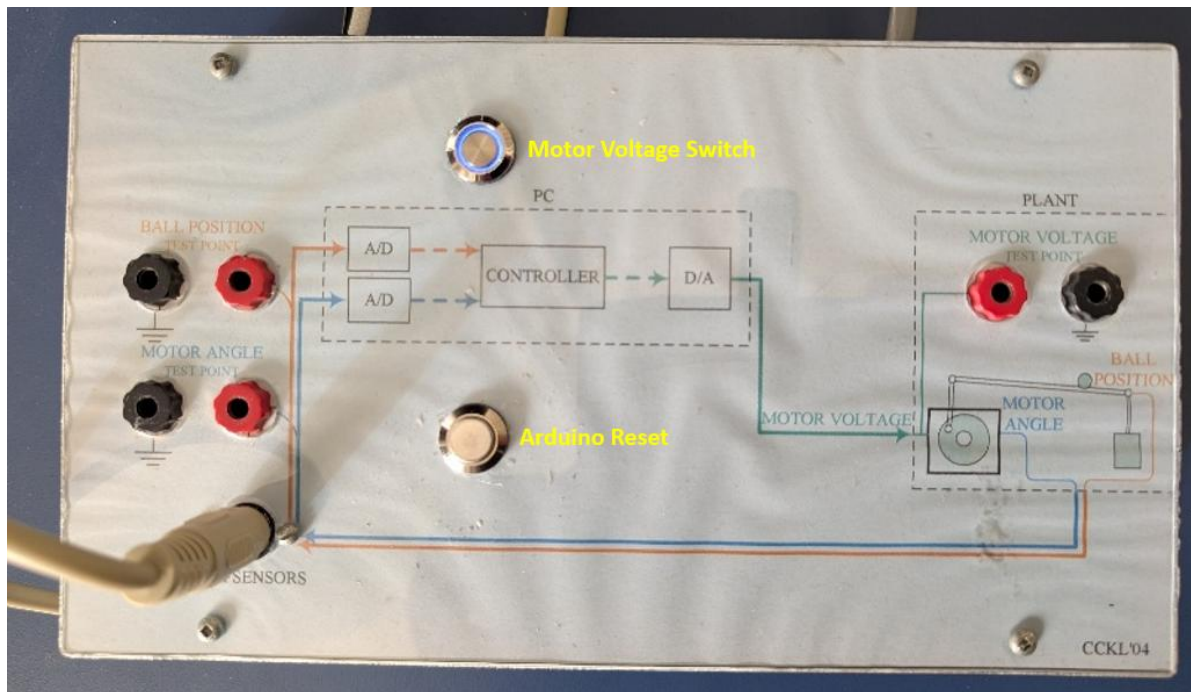
If the Arduino UNO R4 Minima option is not available you will need to include it via the “Tools → Boards” Menu.

Search for the keyword Minima in the menu search bar and install this board in your Arduino IDE.

Your Arduino code can then be compiled and transferred to the system using the IDE controls indicated below. The Transfer option will compile your code and then transfer it.



To transfer code to the Arduino controller, press the Reset Button on the Arduino Box twice. After the second beep of the system, the code may be transferred by selecting the Transfer icon in the IDE (→).



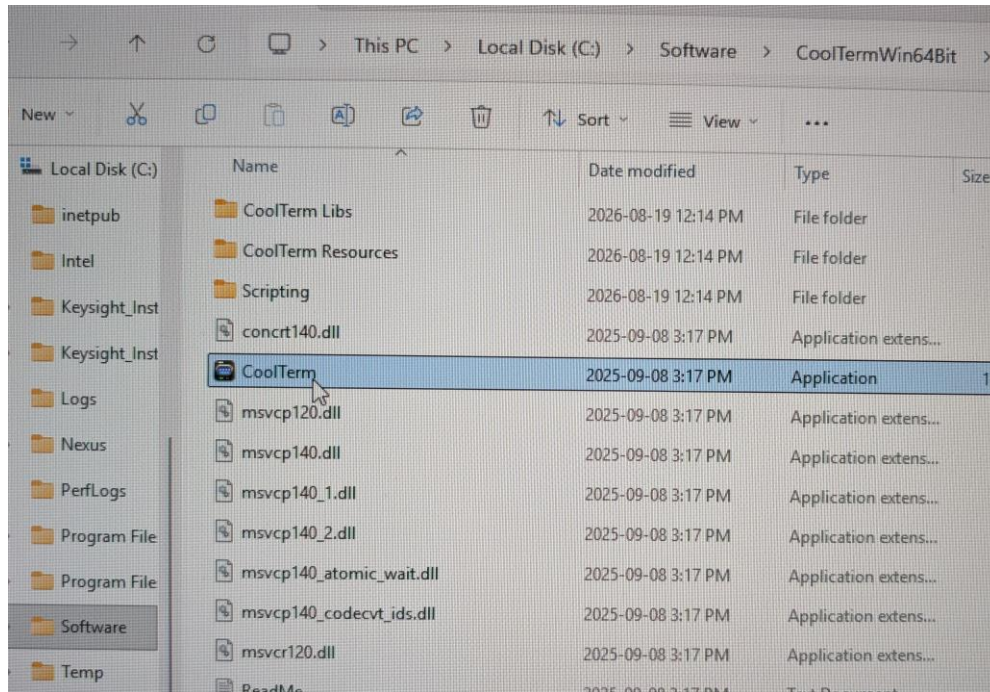
The motor voltage switch, when lit, indicates that power from the Arduino box motor controller can be applied to the motor.

If necessary, in an emergency, power can be removed from the motor by actuating the motor voltage switch on the Arduino box so that it is taken to the un-lit state.

This may be very useful if the control system does not operate as intended and causes the beam connection assembly to jam against the gears of the servo unit. If this happens, the motor voltage switch can be used as an emergency stop switch.

Once your Arduino code is ready, data collection can be done using the app CoolTerm. This app intercepts all information transferred via the Arduino Serial Port, displays it on its main window and can save the data to a text file.

You will need to download the CoolTerm compressed file available on LEARN and extract the files onto your Documents folder to use CoolTerm in your personal computer (Figure 3 folder structure). The lab computer stations have Coolterm installed as shown in the directory structure below.

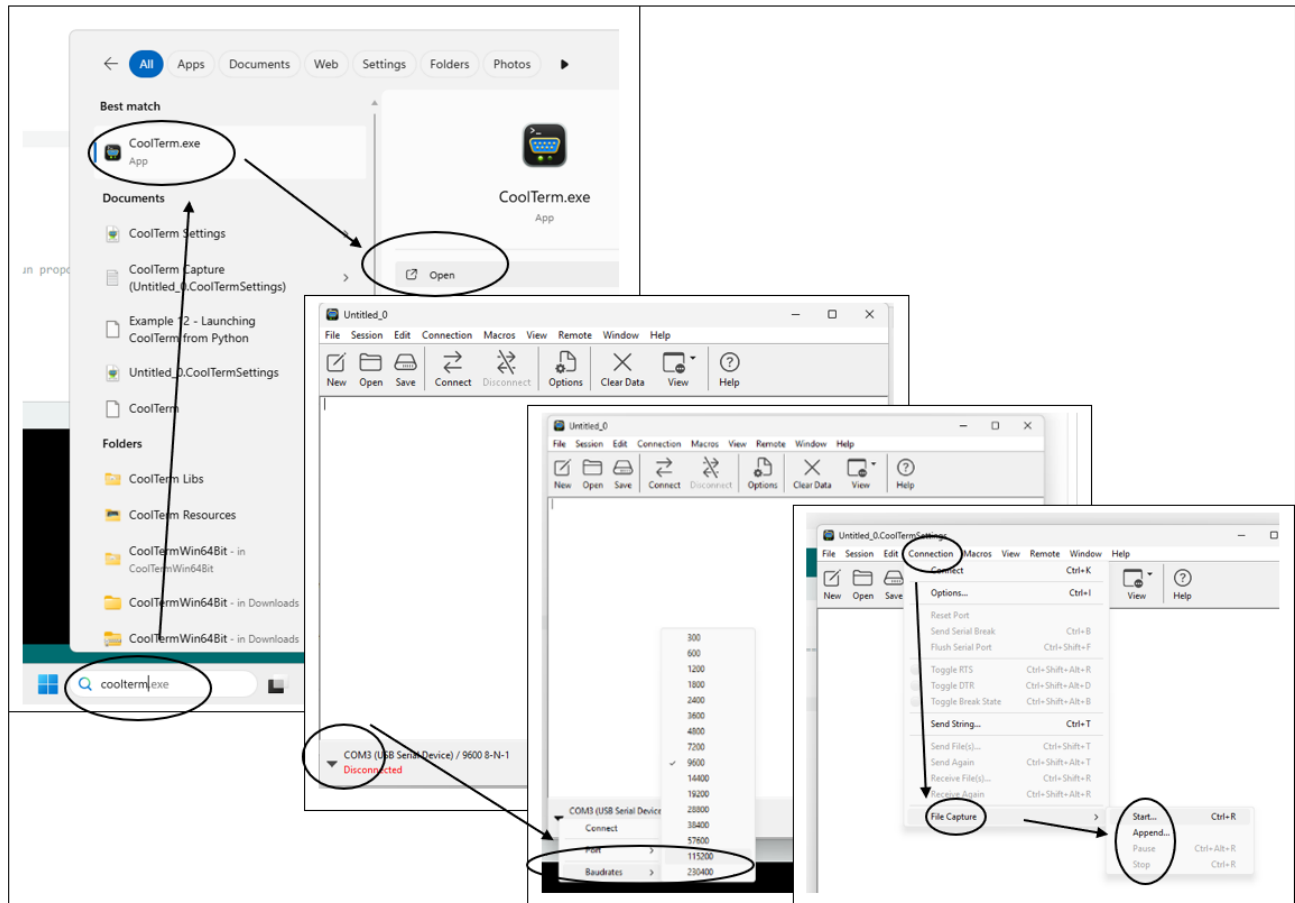


You can generate any set of data by including the necessary `Serial.print()` and `Serial.println()` commands in your Arduino code to monitor variables during program execution.

The sample code provided implements a simple csv-style logging just as an example.

To allow CoolTerm to capture the data from the Arduino serial port, close the Serial Monitor window in the Arduino IDE.

Once CoolTerm is launched, adjust the baud rate and activate data logging to file via the Connection menu option. The data can be copied and pasted onto Excel.



At your discretion, you can also use Matlab to acquire data from your Arduino into Matlab to use its rich processing and plotting capabilities.

Another option is to stream your real-time data directly into Excel using the Excel Data Streamer Add-in.

Performing measurements on system responses will be required throughout the labs, so the groups are advised to early on establish a graphing and measurement strategy.

Before attempting to control the ball and beam system, it is important to consider the control structure; that is, decide if a single control loop or an inner/outer-loop control structure is best, and consider what type of compensation is reasonable. The essential linearized ball and beam dynamics are summarized in a simplified model as follows:

$$\frac{\theta(s)}{V(s)} = \frac{K_1}{s(\tau s + 1)} \quad \text{Motor Gear Angle to Voltage Relation ... (1)}$$

$$\frac{\phi(s)}{\theta(s)} = K_2 \quad \text{Beam Angle to Gear Angle Relation... (2)}$$

$$\frac{Y(s)}{\phi(s)} = \frac{K_3}{s^2} \quad \text{Ball Position to Beam Angle Relation... (3)}$$

where signals $V(t)$, $\theta(t)$, $\phi(t)$, and $y(t)$ (and some key geometric variables) are shown in Figure 4. Notice, among other details, the direction of measurement for the motor gear angle and for the ball position on the beam.

The plant parameters K_1 , K_2 , K_3 and τ will be determined as you progress through the lab activities, while other specifications of the ball and beam apparatus are available in [Appendix 1](#).

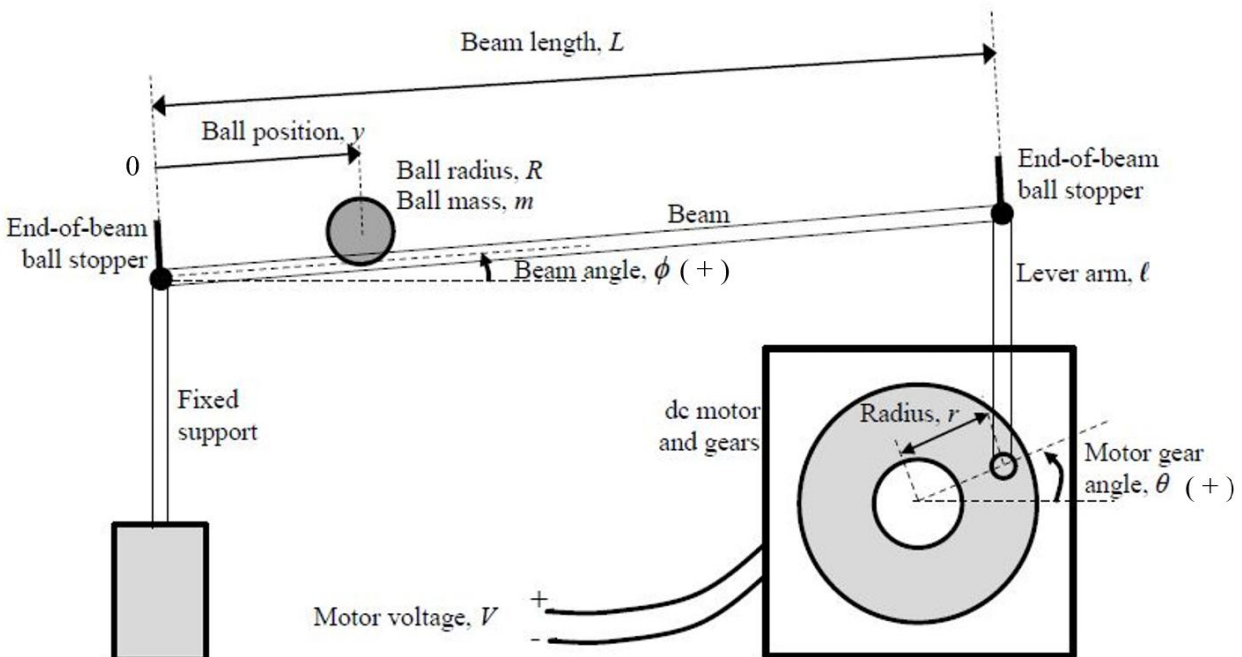


FIGURE 4: Geometry of the ball and beam system.

The simplest control topology is a single feedback control loop shown in Figure 5, where the plant is treated as the cascade connection of (1), (2) and (3). Although it is possible to design the compensator $C(s)$ such that the closed-loop system is stable, the triple integrator in the plant contributes 270° phase lag to the loop gain making it difficult to obtain “good” closed-loop performance.

Another practical difficulty with this setup is that it is not clear how one can ensure that the motor does not rotate so much as to damage the lever arm mechanism.

To avoid damage, we must ensure that

$$-\frac{\pi}{4} \text{ rad} < \theta(t) < \frac{\pi}{4} \text{ rad}$$

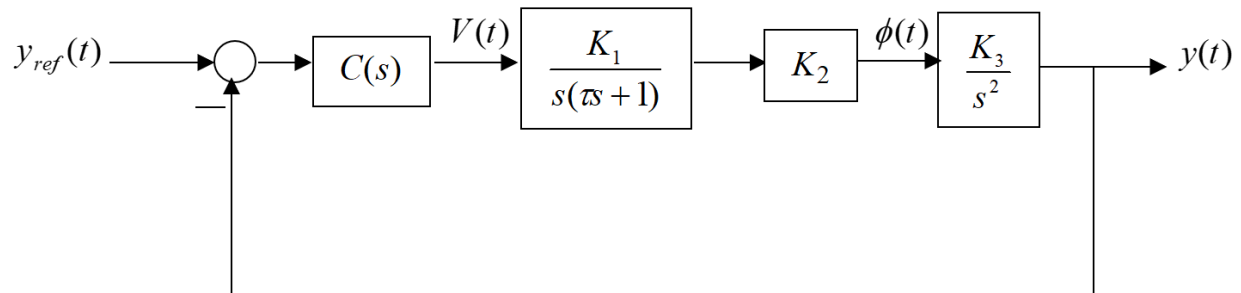


FIGURE 5: A single feedback control loop for the linearized plant. Note that a continuous-time controller is used in this diagram even though a sampled-data controller would be used in the experimental implementation in the lab.

An alternative control arrangement is shown in Figure 6. In this setup, two feedback loops are used.

The purpose of the inner-loop is to control the motor gear angle position; controller $C_1(s)$ should be designed so that $\theta(t)$ tracks the reference signal $\theta_{ref}(t)$.

The outer-loop uses the inner feedback loop to control the ball position. From the point of view of controller $C_2(s)$, the “plant” is composed of everything enclosed by the dotted box in Figure 6.

Clearly the inner-loop must be designed before the outer-loop for this cascade control configuration.

We will also see that the inner/outer-loop control setup in Figure 6 avoids the two problems associated with the single loop control configuration in Figure 5.

Although the block diagrams in Figures 5 and 6 show continuous time controllers, you will ultimately design and implement discrete time controllers.

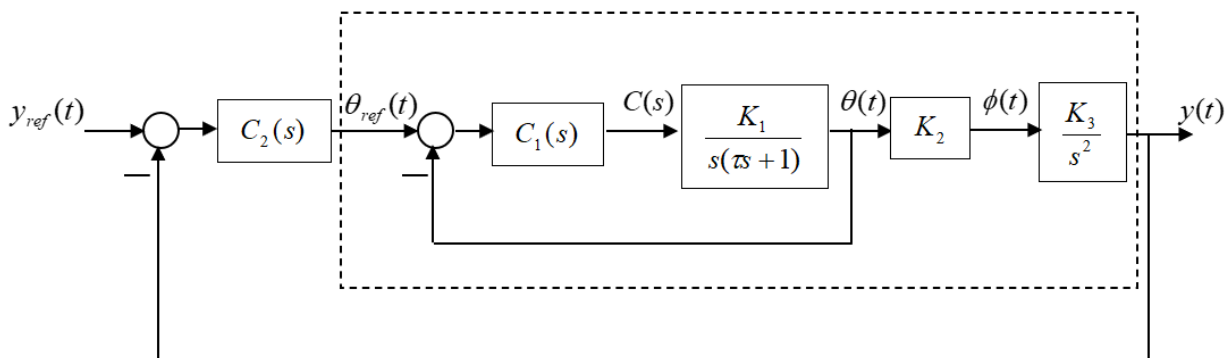


FIGURE 6: Inner/outer-loop control configuration for the linearized plant. As in Figure 5, continuous-time controllers are used in this diagram.

GENERAL REPORT REQUIREMENTS

▪ **Report Submission**

- Submit your lab report to **Crowdmark**:
 - Upload the parts of your report to each corresponding question as instructed in the Crowdmark assignment.
 - Unless explicitly instructed to do so, never submit all your work in one question. This hinders proper evaluation and grading.
 - Each lab manual question/part has its dedicated section in the assignment.
 - Make sure you submit your answers to each question/part correctly to avoid unnecessary deductions to your Lab Report mark.
- Submit your lab report and any additional deliverables requested to **LEARN**:
 - Attach a pdf format file containing your full lab report to the appropriate drop-box.

▪ **Report Write Up**

- Include the signed Authorship Declaration.
- This should include your lab group number and the station number you have used.
- The station number should be the same as the station initially assigned to your lab group, unless a change was previously and formally approved by the lab instructor.
- **Report should be a self-contained and complete narrative of the work performed.**
- As required, comment on every lettered section from the lab manual and follow the sequence informed in the Crowdmark assignment.
- Lab Reports must be typed, with handwritten parts allowed for diagrams and formulas.
- Discussions of results and descriptions of procedures used to design your controllers and conduct the experiments must be typed.
- In any case, handwritten parts must be easily readable and kept to the minimum.
- Handwriting the entire analysis is not acceptable.
- Utilize a suitable font size, no smaller than 12pt Times New Roman, and single line spacing, clear units, clear lines and labels, and ensure your graphs and screenshots are crisp and legible.
- Indicate performance measurements on your plots using cursors and/or labels and, if necessary, with "zoomed-in" snapshots or separate plots to confirm your assertions.
- If your report features block diagrams, consider consolidating the individual blocks into a single figure. Distinguish them using colors, line types, and clear legends. You are not limited to any specific plotting interface, just remember that anything that cannot be read effectively may result in lost marks.
- It is a known Crowdmark problem to sometimes experience loss of resolution when uploading graphics.
- Possible measures to avoid this:
 - When including/uploading graphics objects to your lab report, avoid using a screenshot of a screenshot.
 - When composing your report, make graphics objects large enough to allow clear reading of the details.
 - When using colors to highlight graphics details, provide good contrast to facilitate reading.

▪ **Report Experiments**

- Describe your thought process comprehensively. Don't merely present the final iteration or just the final result.
- Describe how you arrived at your conclusions and the reasoning behind your choices.
- Show your data, don't just mention it. Results stated in the report must be supported by data obtained from experimental measurements, established controls theory or math & science concepts.
- For example:
 - "We Need to Do This. We began with 1, 2, 3, and discovered/obtained a, b, c.
 - Therefore, because of d, e and f, we went with 4, 5, 6 and achieved x, y, z, which can be seen in the following plots and data tables.
 - X and y perfectly align with expected results.
 - Z does not align with expected results. This can be due to AA and BB."

▪ **Report Data**

- Include enough data and information so that your results are reproducible. Data used to obtain results stated in the report must be included in the Lab Reports, not just mentioned or hinted at or implied.
- Indicate units when giving numerical data and clearly define all variables used.
- Screenshots included in Lab Reports need to support the results stated and/or the procedures followed while performing the experiment.
- Captions must be included to inform the purpose of the screenshots.
- Legends must be added to the screenshots to identify the plot traces when legends are not automatically generated by the software being used.
- Use of cursor lines must be made to clearly indicate measurements being performed in the screenshots.
- If necessary to support conclusions in the lab report, explicitly indicate any arithmetic operation being performed using numerical values taken from the screenshots, including those related to the cursor lines.
- In general, actual experimental system data will not exactly match simulated data.
- Point out unexpected results and provide possible explanations.
- Discuss any differences:
 - Indicate where they came from.
 - Indicate whether or not they are significant enough to pose a problem.
 - If appropriate, indicate what you could do to mitigate associated problems.
- This discussion should be a qualitative analysis, not quantitative.
- Do not include pages and pages of percent error calculations.
- Discuss the validity of any assumptions made.

- **Final Comments**
 - Formatting and presentation are important. Make your report readable and easy to follow. For example:
 - Do not put all data in appendices.
 - Do not put all figures at the end of your report.
 - Use appendices judiciously to organize your work and avoid unnecessary repetition.
 - For example, calculations should be included within the report.
 - If calculations are repetitive, display them for one case in the main body of your report, in the pertinent question, and mention in the text that you applied the same method for other cases, which can be included in an appendix at your discretion.
 - Quality matters more than quantity. Do not put fillers in your report.
 - Include References.

LAB 1: MODELING OF THE MOTOR SYSTEM

This lab manual is written such as to allow students to exercise their engineering judgement and creativity. Design objectives must be met, but how you complete various tasks can vary from one lab group to another.

For Lab 1, we focus on the inner-loop in Figure 6, where the motor gear angle is controlled. The loop is duplicated in Figure 7(a); Figure 7(b) shows the later addition of a saturator to limit the motor operating angle range needed when connecting the beam; Figure 7(c) shows the discretized controller acting on the continuous plant.

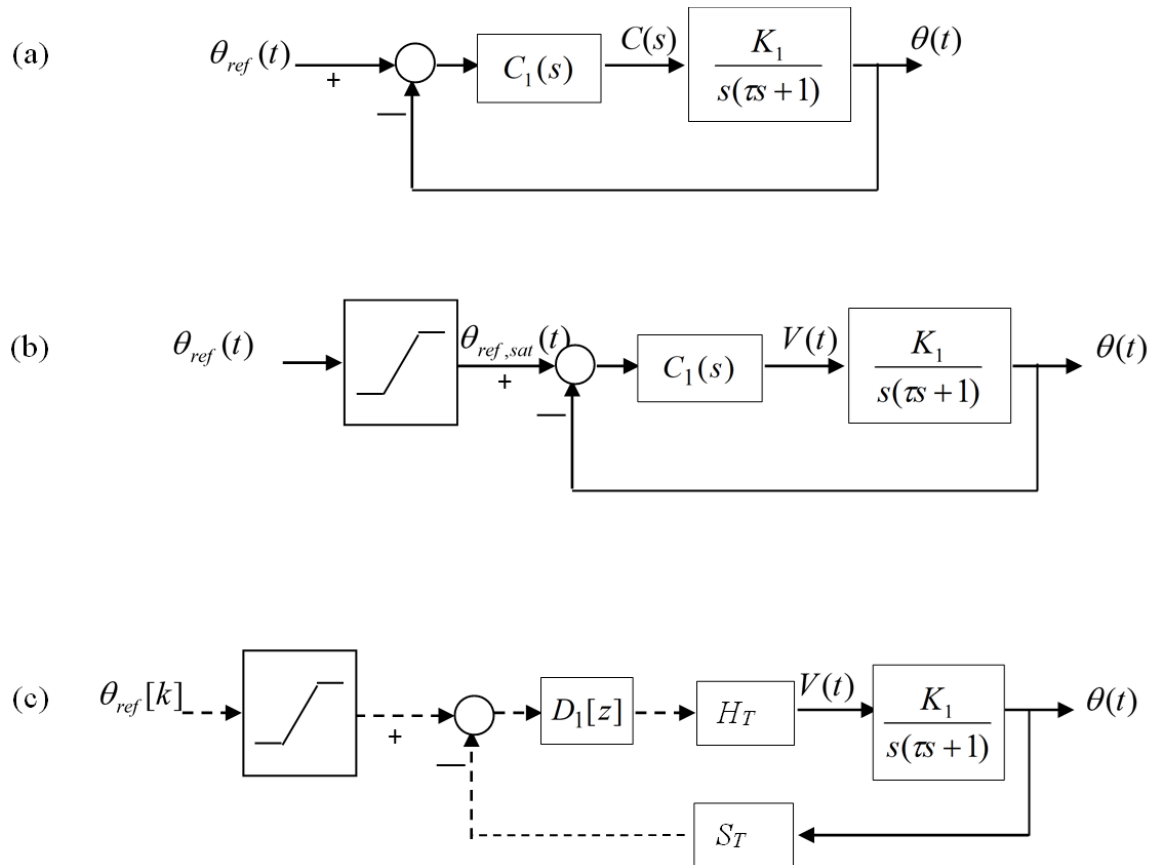


FIGURE 7: (a) continuous-time inner loop, (b) continuous-time inner loop with a saturator (c) sampled-data inner loop with a saturator (the dashed arrows represent discrete-time signals, the S_T block represents an ideal sampler, and the H_T block represents an ideal zero-order hold)

Warning:

Make sure that the beam **is not attached** to the gear at this point and throughout Lab 1. You need a working controller for the inner-loop to avoid damaging the ball and beam apparatus!

- (a) Familiarize yourself with the Arduino and Ball and Beam System. Set up your programming environment as early as possible. It is strongly suggested that BOTH partners set up the Arduino environment to be able to compile the files on their own, and keep a copy of ongoing project work, as a measure of redundancy, and availability of files for sharing with lab partner and for submissions and demo session.
- (b) Check out your experimental station. Figure 8 shows a detail of the motor system plant and how to correctly measure the gear angle θ . Use the sample code provided and exercise the system to cause rotation in both cw and ccw directions. Use the Serial Monitor and CoolTerm app to display the data being sent to the serial port. Save the CoolTerm data to a file. Import the data into MS Excel and plot it. This will prepare you for items (c) and onwards as you progress through the lab activities. Observe the signals being read from the two system sensors. Are the signals “clean” or impacted by electrical noise? How can you apply filtering to the signals if required? Considering these aspects early on will be a great benefit to your project.

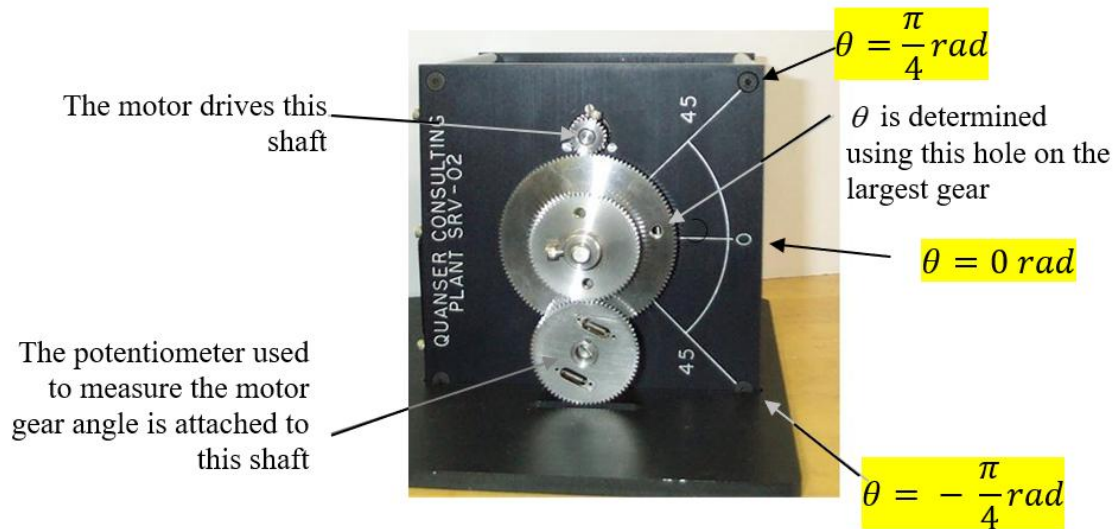


FIGURE 8: The motor system plant for Lab 1

- (c) The motor gear angle is measured using a potentiometer. It is necessary to convert the potentiometer reading to the corresponding gear angle in **radians**. This relationship is nonlinear due to an offset, as shown in Figure 9. You should invert this nonlinearity in software by performing the sensor scaling so that the overall system is linear. To implement the gear angle scaling, use readings relative to the three angles highlighted in yellow in Figure 8. Then, write the necessary code in the Arduino application to compute the gear angle in radians. Expected format for the gear angle sensor scaling is an equation of the type:

$$\text{rad} = m * \text{sensor reading} + \text{offset}$$

You can use the sample code provided for this task, to obtain the A/D reading at each of the 3 angle positions. Having these 3 values and their corresponding radians values will allow you to linearize the gear angle reading from the potentiometer. Document your results in your lab report.

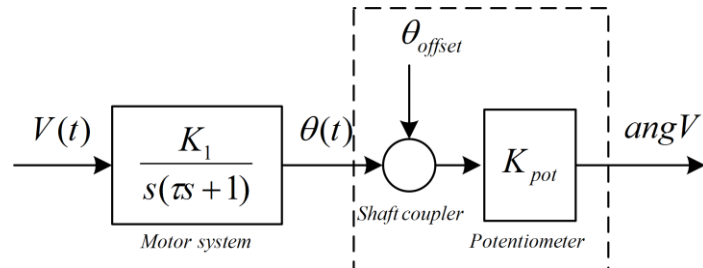


FIGURE 9: Nonlinearity introduced by servo-angle potentiometer

(d) Any motor has some amount of static friction, often called “stiction”. From a control point of view, stiction is undesirable since it can make fine motor motions very difficult. One strategy to deal with stiction is to (approximately) cancel it out:

- Experimentally determine the motor voltage required to overcome stiction. The following tips may help:
 - You may use the sample code provided to determine the “motor stiction” or, to avoid repeated code compilation, you may modify that code to implement a less manual approach.
 - The motor stiction offset should be selected so that, ideally, the dead zone is completely eliminated.
 - Stiction may be nonsymmetrical, so you need to determine the stiction offset for each direction: clockwise and counter-clockwise.
- The term direction, clockwise or counter-clockwise, refers to the direction of rotation of the large gear, viewed from the front.
- Stiction compensation will need to be implemented in your Arduino code.

In your Lab 1 report, briefly explain how you determined suitable stiction offset values and report the values found.

From this point on, it will be assumed that the stiction compensation is implemented in your experimental setup, using the values you determined for motor stiction.

- (e) If you are not familiar with DC motor models, consult an undergraduate control textbook to verify that a standard model for a motor position control system can be expressed as:

$$\frac{\theta(s)}{V(s)} = \frac{K_1}{s(\tau s + 1)}$$

Using an experimental method, determine the specific values of K_1 and τ for your lab apparatus (plant). Note the following:

- The plant is open-loop unstable, and therefore it is difficult to model the plant directly. Since, at this point, we have not designed any controller, stabilize the plant by implementing the system suggested in Figure 7(a), with $C_1(s)$ implemented by a proportional controller.

Use the standard format for a second order transfer function.

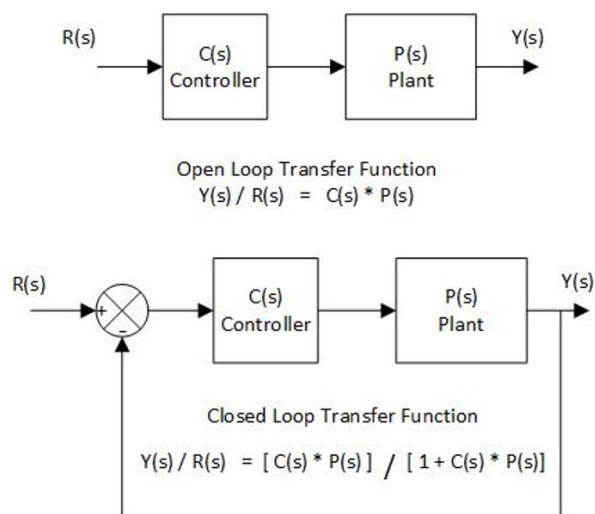
Observe the relation of Damping Ratio (ζ) to Overshoot (OS), and of Natural Frequency (ω_n) to Time-to-First-Peak (T_p).

Making experimental measurements of OS and T_p from the closed loop system response to step inputs, you will be able to deduce the unknown system parameters to determine the model of your **closed-loop** system and compute K_1 and τ .

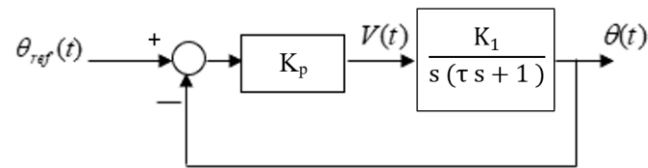
Pay special attention to signs, as some values may be negative.

The following refresher information may also help perform the system identification.

Open Loop Transfer Function & Unity Feedback Closed Loop Transfer Function



For our system:



We know that

$$\frac{\theta(s)}{\theta_{ref}(s)} = \frac{\frac{K_p K_1}{s(\tau s + 1)}}{1 + \frac{K_p K_1}{s(\tau s + 1)}}$$

Which is a second order system, therefore:

$$\frac{\theta(s)}{\theta_{ref}(s)} = \frac{\frac{K_p K_1}{s(\tau s + 1)}}{1 + \frac{K_p K_1}{s(\tau s + 1)}} = \frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2}$$

And for Second Order Systems, ζ and ω_n can be calculated from the system response:

Time Response Formulas For 2nd Order Systems

$$G(s) = \frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2}$$

$$\zeta = \frac{-\ln(\frac{\%OS}{100})}{\sqrt{\pi^2 + (\ln(\frac{\%OS}{100}))^2}}$$

$$T_p = \frac{\pi}{\omega_n \sqrt{1 - \zeta^2}} \rightarrow \omega_n = \frac{\pi}{T_p \sqrt{1 - \zeta^2}}$$

$$T_s = \frac{4}{\zeta \omega_n}$$

- Experimentally determine a sampling rate that is sufficiently fast to avoid loss of data during the system identification experiment. In your report, indicate what sampling rate you used and why.
- With a stable inner-loop, verify that the stiction value you selected in (d) is adequate: the stiction offset should be high enough so that the first peak of the step response is well rounded with no flat portions, but low enough that the step response settles to its steady-state value with no major gear oscillations. Another check is that there should be little or no steady-state error.
- Use 3 sets of data with different controller gains to increase the probability that your system identification is accurate. Use the same constant small step size for each data set. Keeping the step size small will reduce the possibility of saturation on the control effort to the motor. We want to ensure that the absolute value of the voltage to the motor during system identification remains below 6 V DC to avoid saturation.
- You can generate step inputs to your closed loop system by generating square wave signals in your Arduino code. Amplitudes to be used for system identification must be low; suggested values are square wave signals going from -0,1 to + 0.1 rad or from 0 to 0.2 rad. Use proportional controller gains that generate a well pronounced overshoot in the system response. Due to station hardware implementation, the proportional gains you will use to perform the system identification are negative numbers.
- **Knowing your plant from a numerical perspective will allow you to design the controllers required in labs 2 and 3 to control the ball and beam system using the theory to be presented in the lecture component of the course.**

The values obtained for the plant identification described above should be in the ranges below:

$$abs(K_1) = 1.50 - 3.00 \text{ rad}/(Vs)$$

$$\tau = 0.010 - 0.035 \text{ s}$$

- (f) To help prevent damage to the Ball and Beam apparatus when the beam is connected to the motor in Lab 2, it is critical that the motor gear angle satisfies

$$-\frac{\pi}{4} \text{ rad} < \theta(t) < \frac{\pi}{4} \text{ rad} \text{ at all times.}$$

To help ensure this, prefilter $\theta_{ref}(t)$ using a saturator, as shown in Figure 7(b).

In Lab 2, you will design a controller to achieve an overshoot no higher than 5%.

When this controller is operating, the condition

$$-0.7 \text{ rad} < \theta_{ref,sat}(t) < 0.7 \text{ rad}$$

will ensure that

$$-\frac{\pi}{4} \text{ rad} < \theta(t) < \frac{\pi}{4} \text{ rad at all times.}$$

Use Simulink to simulate the system used in part (e) in the configuration shown in Figure 7(b) to verify that the saturator works as expected. Use a square wave generator to generate the input reference to the Simulink system. Include in your Lab 1 report the Simulink block diagram used and a simulated plot showing the performance of the saturator for positive and negative angles including saturator input and output on the same plot.

Test both positive and negative angle references.

The K_1 value you obtained is a negative number and the proportional gains used were also negative numbers. In Simulink, at your discretion, you can use both as positive numbers for your simulation.

Perform an experiment to show that the saturator is functioning as desired, in the configuration used to perform the system identification. Include a plot of the saturator performance in your Lab 1 report, with saturator input and output on the same plot.

Test both positive and negative angle references.

Warning: Do not attach the ball and beam to the motor at this point.

(g) Draw a block diagram for the inner-loop showing the state at the end of Lab 1, including:

- All blocks used
- Experimental signals with correct signs
- Variable names used in the Arduino code, and their units
- Nonlinear effects, such as offsets and saturation
- Sensor scaling
- Distinguish (color or line type) between analog (continuous time) and digital (discrete time) signals
- Indicate which blocks, specifically, are part of the apparatus

Note 1: When choosing the platform for creating the block diagram, keep in mind that in Labs 2 and 3, as the scope of the project expands, you will add to this diagram, which will again be part of your deliverables in Labs 2 and 3.

Note 2: About Block Diagrams:

Block Diagrams show relation from Input to Output, ideally in a straight sequence of blocks from left to right, with system input represented in the left-most block and with system output represented in the right-most block.

Inputs to Block Diagrams usually represent a desired state – Set Point or Reference.

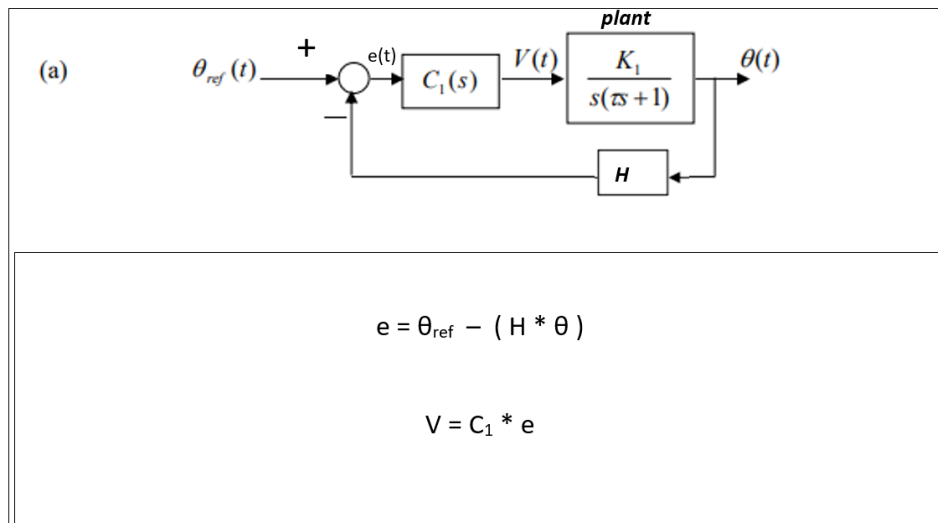
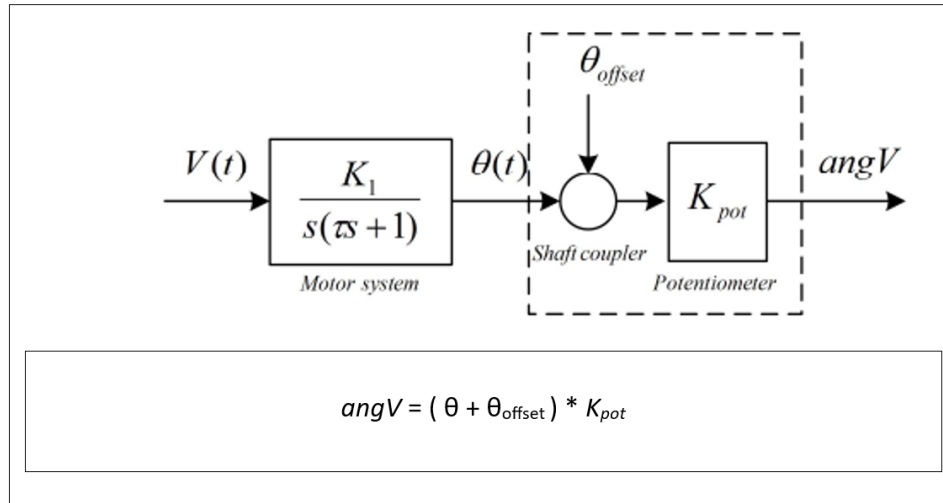
Outputs from Block Diagrams usually represent the work expected from the system.

Sensor output signals are usually not the desired output from the system.

Sensor output signals are a means of measuring the desired physical output from the system.

Sensor output signals are used as feedback and represented in feedback loops.

Note 3: Coding from Block Diagrams:



(h) Include your Arduino code(*.ino) in your report and upload it to the Lab 1 Dropbox.

In addition to the [General Report Guidelines](#), please keep your report as brief as possible.

LAB 2: DESIGN OF THE INNER-LOOP CONTROLLER AND MODELING OF THE BALL AND BEAM

INNER-LOOP CONTROLLER DESIGN

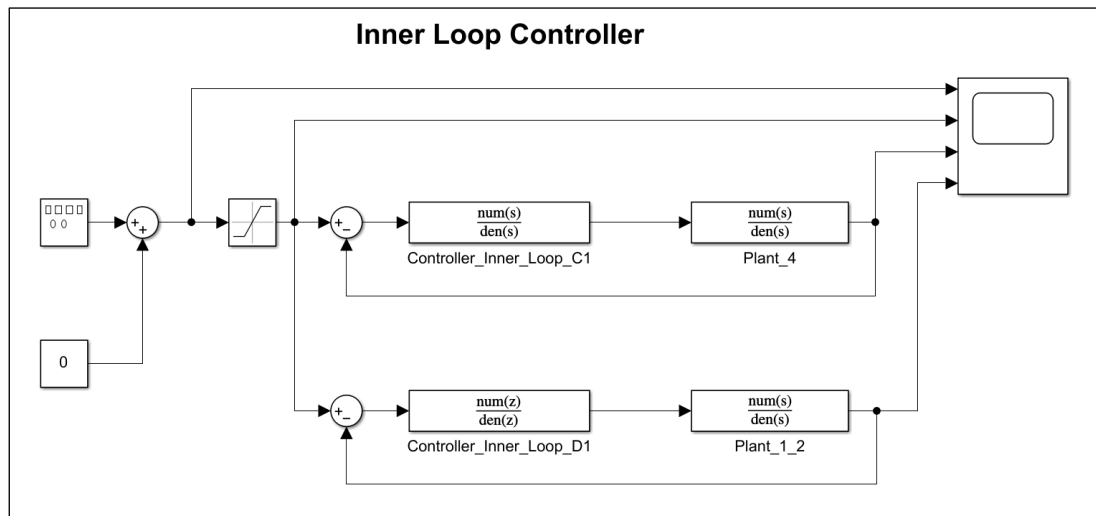
(a) We will be using the emulation design approach for Lab 2. **Design a continuous time controller $C_1(s)$** in Figure 7(a) to satisfy **in simulation** the following specifications:

- The (linearized) feedback loop is stable
- When the reference signal steps from $\theta_{ref} = -0.7\text{rad}$ to $\theta_{ref} = 0.7\text{rad}$:
 - The step response steady-state tracking error is zero
 - The step response 2% settling time is no more than 0.25 seconds
 - The step response overshoot is no more than 5%
 - The motor voltage does not exceed $\pm 6\text{V}$
- Your controller must be at least first-order, so that you can apply, in parts (b) and (c), concepts from this course. A pole-placement design or classical lag compensator design from your pre-requisite controls course may be used. Use of MATLAB, or similar, tools such as Control System Designer, Control System Tuner and PID Tuner are allowed as a means of tuning your controller, however MATLAB or similar or alternative tools cannot be used plainly in a trial-and-error approach to obtain the controller $C_1(s)$.
- State and document your $C_1(s)$ controller design and its performance in your lab report.
- Use a reference step input with a 50% duty cycle and period of 2 seconds.

Verify the effectiveness of your controller in simulation, but do not try it out on the real experiment yet. If your controller does not work as expected, one possible reason could be an incorrect sign.

(b) Once your continuous time controller works as intended in simulation, discretize it.

- Choose a suitable sampling period, state it and provide justification in your lab report.
- Use any reasonable discretization method to find the discretized controller, $D_1[z]$, and justify it in your lab report. State your discrete time controller in your lab report. Use Simulink to simulate the sampled data system as shown in Figure 7(c). Confirm that the closed-loop performance in discrete time is similar to that of the system in Figure 7(b) in continuous time, and document this in your lab report. Include in your lab report your Simulink block diagram and performance plot, showing reference input, continuous time controller output and discrete time controller output on the same plot.



Note that if you make $D_1[z]$ a discrete transfer function, then Simulink will automatically insert the sample and hold operators shown explicitly in Figure 7(c).

Do not insert your own sample and hold, and be sure to correctly set the selected sampling period in the discrete transfer function.

- (c) Implement your controller $D_1[z]$ on the real experiment, **with the beam disconnected from the gears**. A guide on how to transform your controller from the discrete time domain $[z]$ to the difference domain for implementation on the Arduino system can be found in [Appendix 2](#). Remember that the sampling period used to design your controller needs to be used to test it on the Arduino system.

It is important to put as many decimals as you can on your controller coefficients; in Matlab, `format long` will give you 15 digits. Numerical errors, associated with rounded up coefficients, may cause your controlled system to go unstable. In Matlab, to extract the coefficients of your controller, `SYS`, you may use `[NUM, DEN]=tfdata(SYS, 'v')`.

Be sure to include the gear angle saturator in your code, as shown in Figure 7(c). This must ensure that

$$-0.7\text{rad} < \theta_{ref,sat}[k] < 0.7\text{rad}$$

Warning!

To avoid damage to the equipment, do not attach the beam to the motor at this point and until your controller for the inner-loop is working satisfactorily, i.e., $-0.7\text{rad} < \theta(t) < 0.7\text{rad}$ and you have the saturator code in place.

The experimental implementation for the inner loop controller must satisfy the following specifications:

- The (linearized) feedback loop is stable
- When the reference signal steps from $\theta_{ref} = -0.7\text{rad}$ to $\theta_{ref} = 0.7\text{rad}$:
 - The step response steady-state tracking error is zero
 - The step response 2% settling time is no more than 0.40 seconds
 - The step response overshoot is no more than 5%
 - The motor voltage does not exceed $\pm 6\text{V}$
 - Use the same sampling period as was used in the simulated controller in (b).

Redesign the controller if the specifications above are not met.

Looking at your system performance plots, you may notice that the motor voltage being output from your control system exceeds $\pm 6\text{V}$.

Experimentally, as long as the amount by which Motor Voltage exceeds $\pm 6\text{V}$ can be approximately attributed to your stiction values, such excess is acceptable.

Use of a voltage limiter or saturator in software to limit the output to the motor is not allowed.

Compare the simulated sampled-data system's response from part (b) to the experimental response from part (c); document the performances and comparisons and explain any differences you observe in your lab report.

BALL AND BEAM MODELING

REFER TO TABLE A1 IN APPENDIX 1 FOR APPARATUS DIMENSIONS

- (d) **New Motor Stiction Values With Beam Attached.** Now that your experimental controller meets the specs outlined in (c), you can connect the beam to the gear. Obviously, attaching the beam to the motor plant will influence the inner-loop model. Technically, the inner-loop should be remodeled, with the beam attached, and a new controller should be designed. However, due to the high gearing of the motor, the model does not change significantly. For simplicity, we will assume that the inner-loop model has not changed. By changing the stiction offsets, you should find that your original inner-loop controller still meets the specifications. Verify that this is so in both directions for the steady state error and include corresponding plots in your lab report. **Be very careful when running as now the beam is attached to the motor gear.**

To protect the system, include a safety check in your Arduino code to ensure that the output to the motor is 0.0 V if the gear angle is > 0.8 rad or if it is < -0.8 rad.

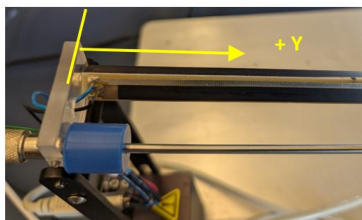
This will ensure that if your controller behaves unexpectedly, the beam attachment will not jam against the gears with power applied to the motor, which can cause the motor to burn. This protection is to remain operational in your code for the rest of your project as you complete labs 2, 3 and Demo.

Document your implementation and test of this feature in your lab report.

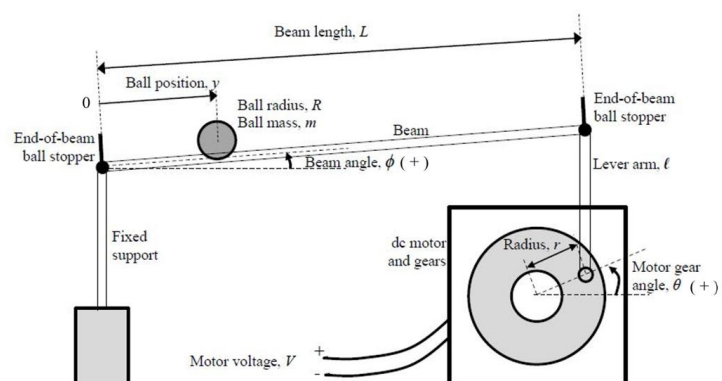
Aspects to ponder to determine new motor stiction values:

- Should the ball be on the beam or not when determining the new stiction offset;
 - Should stiction in both directions change or only in one direction. Explain your choices.
- (e) Convert the ball position reading to the corresponding **effective position in meters** using the y-origin convention shown in **Figure 4**. Show in your Lab 2 report the scaling equation for the ball position, and explain how you determined it. Then, write the necessary code in the Arduino application to compute the ball position in meters. Expected format for the ball position scaling is an equation of the type:

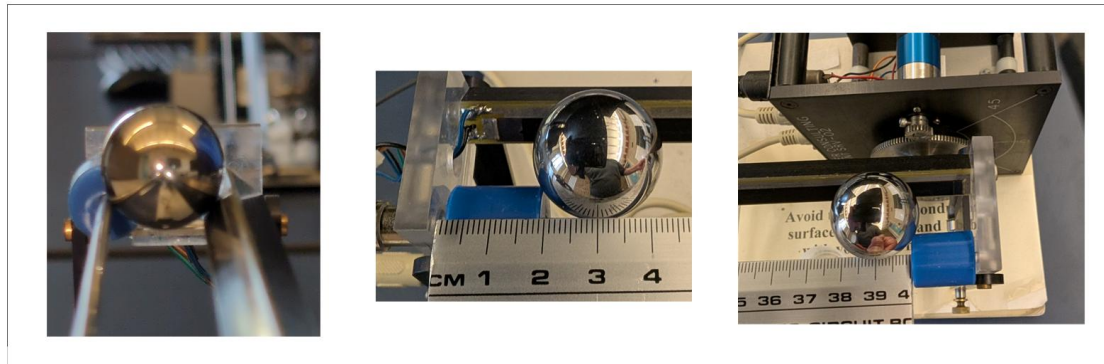
$$\text{meters} = m * \text{sensor reading} + \text{offset}$$



Repeat of FIGURE 4:



While performing the scaling of the ball position sensor, think about how the ball position is effectively sensed, and about the effective positions that the ball can assume at the extremes of the beam.



- (f) A model for the lever arm and the beam, with the structure shown in equations (2) and (3), is provided below:

$$\frac{\phi(s)}{\theta(s)} = K_2 \quad \text{Beam Angle to Gear Angle Relation... (4)}$$

$$\frac{Y(s)}{\phi(s)} = \frac{K_3}{s^2} \quad \text{Ball Position to Beam Angle Relation... (5)}$$

Using geometry and small angle approximations, the value of K_2 is found to be **0.0609**. This is based on the assumption that a successful controller will for the most part operate with small gear angles.

Perform experimental tests to determine the value of K_3 .

Re – arranging (4) $\frac{Y(s)}{\phi(s)} = \frac{K_3}{s^2}$ we obtain

$$s^2 * Y(s) = K_3 \phi(s)$$

1. What does this tell us in the time domain?
2. The generic kinematics equation for displacement is $y(t) = y_0 + y'_0 * t + \frac{1}{2} * a * t^2$
3. How can we work conduct experiments on the ball and beam station to obtain K_3 ?
4. Can your inner loop controller help you in conducting these experiments?

Provide, in your report, description and evidence of the experiments performed to determine K_3 . Multiple measurements are necessary for determining a reliable K_3 value. As a minimum, use 3 positive and 3 negative beam angle values for this purpose.

For determining K_3 values, consider the expected ranges for angles Θ and ϕ of a successful controller in operation.

- (g) Draw a block diagram for the system which contemplates all the work performed so far, as we have the system at the end of Lab 2, after working on points (a) to (f). Show the experimental signals, the variable names used and their units. Be sure to include nonlinear effects, such as offsets and saturation, as well as sensor scaling. Correct signs should also be shown. Indicate which blocks, specifically, are part of the apparatus. Distinguish (colour or line type) between analog and digital signals.

Write up your results for Lab 2 following the [General Report Requirements](#); also provide:

- A table with the values from Lab 1 for K_1 , τ , stiction values, scaling of the gear angle.
- Provide a short justification for any changes made.

Submit your final Arduino code file to the Lab 2 dropbox in LEARN, so that it can be tested by the LI/TAs.

Submit your SLX file, and associated m-files, to the Lab 2 dropbox in LEARN, so that they can be tested by the LI/TAs.

LAB 3: DESIGN OF THE OUTER-LOOP CONTROLLER

In Lab 3, we focus on controlling the ball position. The basic setup is the inner/outer-loop control configuration shown in Figure 6 and repeated below for convenience.

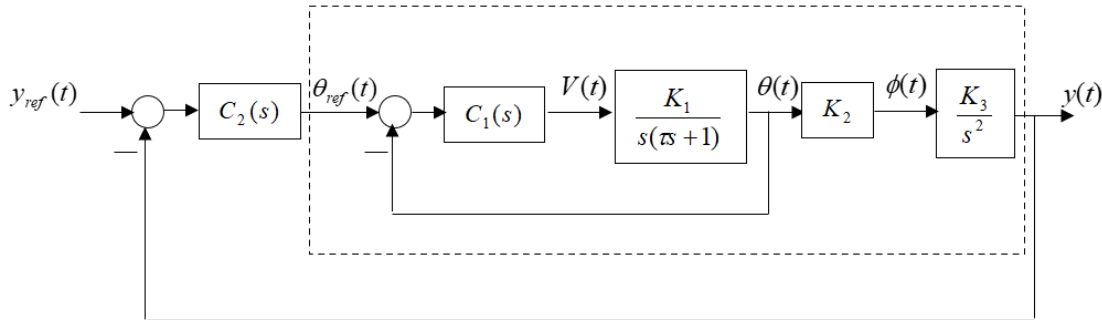


FIGURE 6: Inner/outer-loop control configuration for the linearized plant. As in Figure 5, continuous-time controllers are used in this diagram.

Note that, from the point of view of the outer-loop controller $C_2(s)$ in Figure 6, the “plant” is the system within the dotted box.

Due to nonlinearities inherent to the apparatus, we may not get the expected performance with a simple controller. In the following steps, we will explore different controllers and configurations; progressive changes will be applied to improve performance. If you prefer a different control architecture that satisfies the **final** design objectives, feel free to implement it instead, while documenting in your lab report its progression and results, however your controller must be at least first order, so that you can apply concepts from this course. A pole-placement design or classical lag compensator design from your pre-requisite controls course may be used. Use of MATLAB, or similar, tools such as Control System Designer, Control System Tuner and PID Tuner are allowed as a means of tuning your controller, however MATLAB or similar or alternate tools cannot be used plainly in a trial-and-error approach to obtain your controller.

- (a) **Simulate**, on your **full plant** model, a 0.15m peak-to-peak square wave (from 0.10m to 0.25m) using the following lead controller:

$$C_{2LD}(s) = -7 \frac{s + 0.35}{s + 2.5}$$

Note that the input to the controller C_{2LD} is in meters and the output is in radians; this controller is based on the assumption that $y=0$ is at the left end of the beam (away from the motor). Use a reference step input with a 50% duty cycle and period of 20 seconds or larger if more time to observe a well pronounced steady state is required. **Include the simulation file, simulation block diagram and performance plot in your submission, showing input reference and ball position output on the same plot.**

- (b) **With the gear angle saturator in place, discretize and implement** the controller from (a) in the lab apparatus, and compare it to the continuous time simulation for a repetitive square wave alternating between 0.10m to 0.25m. Document your implementation and its performance in your lab report. **Your experimental plot must include the reference input and the ball position output on the same plot.**

Caution: Be sure to avoid feeding back outputs from nonlinear blocks, such as saturators. Instead, keep nonlinear values in separate variables from those being fed back in your controller (via shift-registers, or other means).

As in Lab 2, it is important to put as many decimals as you can on your controller coefficients; in Matlab, `format long` will give you 15 digits. Numerical errors, associated with rounded up coefficients, may cause your controlled system to go unstable. To extract the coefficients of your controller, `SYS`, you may use `[NUM, DEN]=tfdata(SYS, 'v')`.

At a minimum we would have expected:

- a symmetrical response, and
- zero steady-state error, since the plant model has a double-integrator.

If either of these characteristics were not observed in the lab, explain why in your lab report. Attributing them to random nonlinearities is insufficient. If you think that nonlinearities are at fault, specify which ones and explain why your choice makes sense. *Hint:* think about why/how an integrator provides tracking and how the location of the integrator might affect your results.

- (c) If necessary, to overcome poor tracking, design a new outer loop controller that has an integrator. Before proceeding with the controller design, look at the inner-loop specifications as compared to the outer-loop. Is there any reasonable approximation you could make? If so, provide explanations for your choices in the lab report and document your design process.

Your controller must be at least first-order, so that you can apply concepts from this course. A pole-placement design or classical lag compensator design from your pre-requisite controls course may be used. Use of MATLAB, or similar or alternate tools such as Control System Designer, Control System Tuner and PID Tuner are allowed as a means of tuning your controller, however MATLAB or similar tools cannot be used plainly in a trial-and-error approach to obtain the controller.

Document your outer loop controller design and its performance in simulation in your lab report.

You can use either the emulation or the direct design approach to design the **outer-loop discrete-time controller**, $D_2[z]$. In either case, design it so that the final sampled-data control system $D_2[z]$ satisfies the following specifications **in simulation**:

- For a square wave input from 0.10m to 0.25m:
 - The step response steady-state error is zero.
 - The step response 2% settling time is no more than 7 seconds
 - The step response overshoot is less than 45%

State and justify your sampling interval for the discrete time outer loop controller in your lab report.

Document your design process in your lab report.

Use Simulink to verify that your controller meets the specifications on the **full plant**.

Include the simulation file, simulation block diagram and performance plot in your submission, showing that the specifications are met and showing input reference and ball position output in the same plot.

- (d) **With the gear angle saturator in place in your Arduino code, implement** the $D_2[z]$ controller from (c) in the lab apparatus and assess the performance for a square wave from 0.10m to 0.25m. Carefully check your implementation and cautiously run the experiment, ready to use the motor voltage switch on the Arduino box as an E-Stop if required.

It is unlikely that all the specifications from (c) will be met, but verify this experimentally before advancing.

Include experimental plots in your report, showing performance against specifications and showing input reference and ball position output in the same plot.

If the specifications are not met, explain what aspects of the specifications were challenging.

As a minimum, you need a controller that performs consistently across several cycles. Document this in your lab report.

Do *not* redesign your outer loop controller at this point if specs are not met.

Remember that we simplified the system to arrive at the transfer functions of the various parts of the system model. This simplification, combined with different controller design approaches, will lead to differences in performance. This happens frequently in industrial applications, and not always will the solution be a complete redesign of the controller. In many cases, addressing particular performance shortcomings will lead to the desired performance. One such case is described below.

- (e) You may have noticed that for small beam angles, the ball does not move; it “sticks”. The ball stiction is somewhat similar to the motor stiction encountered in the inner-loop. The strategy used there was to approximately cancel it out by adding a small voltage offset to the motor signal. A suggested configuration to overcome the ball stiction is shown in Figure 10.

The anti-stiction block can encompass any strategy you chose to apply.

It may require additional experiments to determine, for example, at which minimum beam angle the ball starts to move in each direction for your lab station, and this in turn may depend on if the ball is close to the 0.1 m or close to the 0.25 m position.

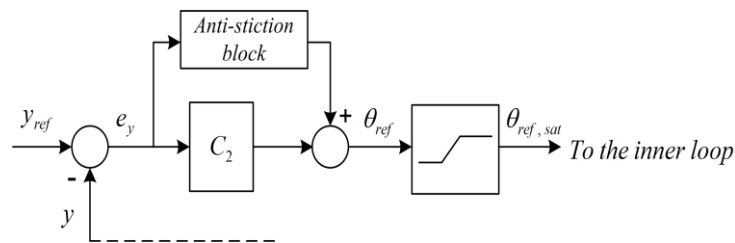


FIGURE 10: A possible ball anti-stiction implementation.

In conjunction with the above strategy, the final control specifications for the outer loop controller are slightly changed as follows:

- The (linearized) feedback system is stable.
- For a square wave input going from 0.10m to 0.25m:
 - The step response steady-state error is such that, at the 0.10 m position, ball position remains within 0.094 m and 0.106 m, and at the 0.25 m position, ball position remains within 0.240 m and 0.260 m
 - The step response settling time to the ranges listed above is no more than 8.5 seconds (enter band and not leave it until after input reference changes)
 - The step response overshoot is less than 45%
 - $\theta_{ref}(t)$ saturation is permitted, because the step size is large, but the gear angle saturator must be present.

If necessary, redesign your outer loop controller to meet the above specifications and document the design process and performance in your lab report.

Performance plots must include traces for the assessment levels detailed above.

Performance against specifications will be assessed across a sequence of 10 consecutive periods of the square wave input.

Sample plot of expected performance assessments can be seen in the section of this lab manual dealing with the demo sessions.

Write up your final lab report using the [General Report Requirements](#); also provide:

- From Lab 1 and Lab 2, restate the values used for K_1 , τ , stiction, scaling of the gear angle, scaling of the ball position, inner-loop controller, K_2 , K_3 and explain if you made any modifications to them.
- Details on how you designed the outer-loop controller, providing a clear justification for the design choices you made, such as to demonstrate course knowledge.
 - If you used emulation, be sure to include:
 - The design of C_2 in Figure 6.
 - Your choice of discretization method.
 - A modified version of Figure 6, clearly showing how your discretized controller is connected. Clearly show all relevant sample-and-hold operators, and include the inner-loop saturator (like in Figure 7(c)).
 - If you used direct design, be sure to include:
 - The mapping of the specifications to the z -plane.
 - The design of the discrete-time controller.
 - A modified version of Figure 6, clearly showing how your discrete-time controller is connected. Clearly show all relevant sample-and-hold operators, and include the inner-loop saturator (like in Figure 7(c)).
- Block diagram of your full implementation.
- Include in your report the Arduino code used.

Submit your final Arduino code including the inner and outer-loop controllers to the Lab3 dropbox in LEARN, along with the Matlab and Simulink files, so that they can be tested by the LI/TAs.

DEMO: DEMONSTRATION OF THE CONTROLLED BALL & BEAM SYSTEM

A formal demonstration of the performance of your B&B controller against the specs listed in 3(e), and, if required, questions regarding your implementation will be scheduled for each lab group after the submission of Lab 3 and before the end of lectures.

Partners can be awarded demo participation marks individually.

If a partner misses their reserved demo time without a valid reason, a mark of zero will be awarded to him/her for the demo. Lateness for the scheduled demo time will also be penalized.

The step from 0.10 m to 0.25 m is to be demonstrated over multiple periods; specifications will be tested on a series of 10 consecutive periods of the square wave input.

Group is allowed to run 11 periods of the square wave input for performance assessment during the demo session.

The code used for the demonstration must be the one included in group's Lab 3 submission and duration of the demo session is limited to 20 minutes per group.

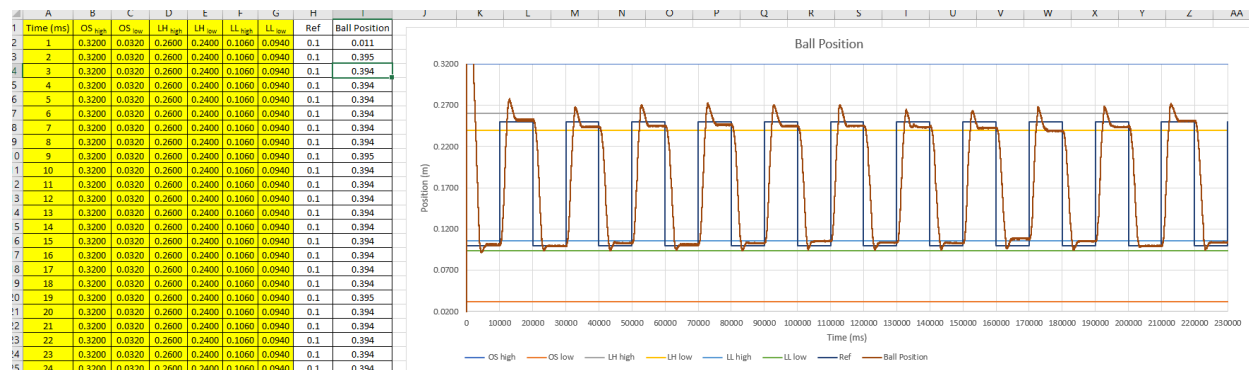
Depending on station availability, Lab Groups may be given 5 minutes in advance of their booked demo session to setup their station and perform an initial test of their controllers.

The demo session will be marked as follows:

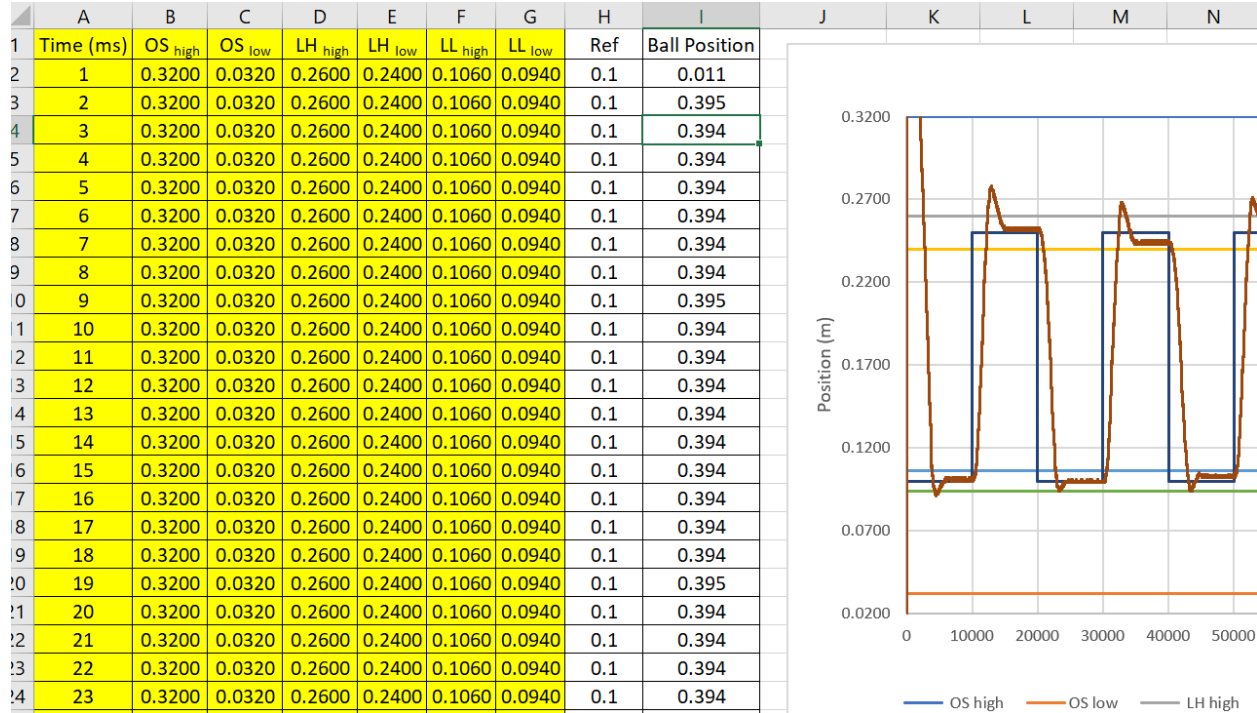
Preparedness – Being on time and having project files to launch the project and assess performance.

Group must come prepared to plot their system performance against the assessment levels detailed below. Sample screenshot follows of a possible plotting alternative using Excel.

1 out of 10 points



The cells highlighted in yellow are to be pre-filled with the assessment levels. Reference input and ball position will be pasted into the file from the live performance run. Time column may be adapted to your specific implementation but must allow clear visual assessment of settling time.



Performance over 10 consecutive periods of the square wave input from 0.10m to 0.25 m

Steady State error - 3 out of 10 points

At 0.10 m, ball position must remain within 0.094m and 0.106m

At 0.25 m, ball position must remain within 0.240m and 0.260m

Settling Time ≤ 8.5 sec for both reference levels – 3 out of 10 points

Overshoot – 1.5 out of 10 points

Moving from 0.10m to 0.25m, ball position must remain under 0.320m

Moving from 0.25m to 0.10m, ball position must remain above 0.032m

Best in class Steady State error – 0.5 out of 10 points

Best in class Settling Time – 0.5 out of 10 points

Best in class Overshoot – 0.5 out of 10 points

BIBLIOGRAPHY

[1] Quanser Consulting Inc., SRV02-Series Ball & Beam User Manual, Revision 1, July 2004

APPENDIX 1

BALL AND BEAM DATASHEET

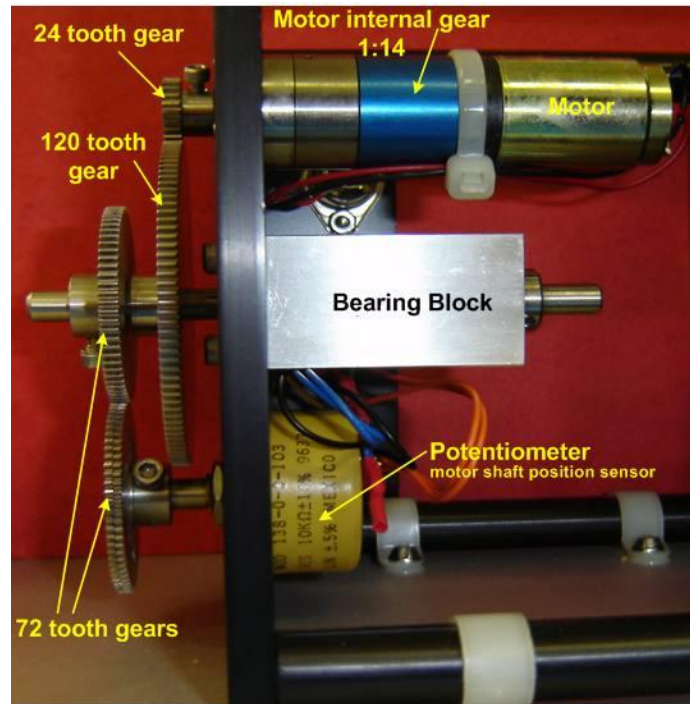


Figure A1: Motor, gear and gear sensor detail

Note that a few generations of ball & beam hardware exist in the lab. Compared to Figure A1, slight variations exist on some stations.

Table A1 – Specifications for the ball and beam apparatus

Specification	Symbol	Value	Units
<i>Motor Subsystem</i>			
Maximum motor voltage	$V_{\max}$	6	V
<i>Ball and Beam Subsystem [1]</i>			
Beam length (interior)	L	41.7	cm
Lever arm length	-	12	cm
Lever end radius of rotation	r	2.54	cm
Support arm length	-	16	cm
Beam inertia (includes motor and gears)	J_{eq}	0.0029	kg m^2
Ball radius	R	1.27	cm
Ball and beam module mass	M_{tot}	0.65	kg
Ball mass	m	0.064	kg

APPENDIX 2

DISCRETE CONTROLLER IMPLEMENTATION

Consider a *generic* transfer function, $T(z)$, which was obtained at a sample period T_s .

$$T(z) = \frac{U(z)}{E(z)} = \frac{a_0 z^3 + a_1 z^2 + a_2 z^1 + a_3}{b_0 z^3 + b_1 z^2 + b_2 z + b_3} = \frac{a_0 + a_1 z^{-1} + a_2 z^{-2} + a_3 z^{-3}}{b_0 + b_1 z^{-1} + b_2 z^{-2} + b_3 z^{-3}}$$

The z -transform operator, z , is a prediction operator; the notation for samples at various time points is shown in Table A2 and can be extended to the number of previous samples needed.

Table A2.

z-operator and Discrete Value	Description	Lecture Notes Notation	Short-Hand Notation
$U(z) \leftrightarrow u[kT_s]$	Current sample	$u[k]$	u
$zU(z) \leftrightarrow u[kT_s + T_s]$	Next sample	$u[k + 1]$	Future value, hence not available
$\frac{1}{z}U(z) \leftrightarrow u[kT_s - T_s]$ $z^{-1}U(z) \leftrightarrow u[kT_s - T_s]$	Previous sample	$u[k - 1]$	u_1
$\frac{1}{z^2}U(z) \leftrightarrow u[kT_s - 2T_s]$ $z^{-2}U(z) \leftrightarrow u[kT_s - 2T_s]$	Second-previous sample	$u[k - 2]$	u_2

Applying the above to $T(z)$, we get:

$$U(z)(b_0 + b_1 z^{-1} + b_2 z^{-2} + b_3 z^{-3}) = E(z)(a_0 + a_1 z^{-1} + a_2 z^{-2} + a_3 z^{-3})$$

$$(u \ b_0 + b_1 \ u_1 + b_2 \ u_2 + b_3 \ u_3) = a_0(e) + a_1(e_1) + a_2(e_2) + a_3(e_3)$$

The output of this transfer function follows:

$$u[k] = \frac{1}{b_0} (-b_1 u_1 - b_2 u_2 - b_3 u_3 + a_0 e + a_1 e_1 + a_2 e_2 + a_3 e_3)$$